
Tribhuvan University
Faculty of Humanities & Social Sciences
OFFICE OF THE DEAN

Digital Logic

BCA — Semester 1 — 2020 — Answer Sheet

Subject Code: CACS105

Group B

Attempt any SIX questions. [6×5 = 30]

2. Define digital logic. Explain digital signal with its applications, advantages and disadvantages.

[1 + 4]

Answer: Digital Logic: Digital logic is the field of electronics and mathematics that deals with digital circuits and systems operating with discrete binary values (0 and 1, or LOW and HIGH). It forms the foundation of modern computers and digital systems, utilizing logic gates, Boolean algebra, and combinational/sequential circuit design.

Digital Signal: A digital signal is a signal that represents data as a sequence of discrete values. It is discrete in both time and amplitude, typically switching between two voltage levels (e.g., 0V for logic 0 and +5V or +3.3V for logic 1).

Applications of Digital Signals:

- Computers and microprocessors
- Digital communication systems (mobile phones, internet)
- Digital audio and video processing
- Medical instruments (ECG, MRI, digital thermometers)
- Industrial control systems and robotics
- Digital measuring instruments

Advantages:

- **Noise Immunity:** Digital signals are less affected by noise because only two levels need to be distinguished.
- **Easy Storage & Processing:** Data can be stored, retrieved, and manipulated efficiently using memory and processors.
- **High Accuracy:** Precision depends only on the number of bits used.
- **Programmability:** Digital systems can be reprogrammed for different tasks.
- **Error Detection & Correction:** Techniques like parity checks and Hamming codes can detect and fix errors.

Disadvantages:

- **Quantization Error:** Converting analog to digital introduces small errors due to finite resolution.
- **Bandwidth Requirements:** Digital transmission may require more bandwidth than analog for the same information.
- **System Complexity:** Requires ADC/DAC converters to interface with the analog world.
- **Clock Synchronization:** Digital circuits require precise timing and clock signals.

3. Define positional number system. Calculate following: a) Subtract 21 from 35 using 2's complement method. b) Convert (62.75)₁₀ into single precision floating point format.

[1 + 2 + 2]

Answer: Positional Number System: A positional number system is a system in which the value of each digit depends on its position relative to the radix point. The position of each digit has a weight that is a power of the base (radix). Common examples include decimal (base 10), binary (base 2), octal (base 8), and hexadecimal (base 16). The general form is:
Value = $\sum (\text{digit} \times \text{base}^{\text{position}})$

a) Subtract 21 from 35 using 2's complement method:

Step 1: Represent both numbers in binary using sufficient bits (6 bits).

35₁₀ = 100011₂

21₁₀ = 010101₂

Step 2: Find 2's complement of 21.

1's complement of 010101 = 101010

Add 1: 101010 + 1 = **101011** (2's complement)

Step 3: Add 35 and 2's complement of 21.

100011 (35) + 101011 (-21 in 2's comp) ----- 1001110 Discard the end carry (leftmost bit). Result = **001110₂ = 14₁₀**.

Verification: 35 – 21 = **14**.

b) Convert (62.75)₁₀ to IEEE 754 Single Precision Floating Point:

Step 1: Convert integer part.

62 ÷ 2 = 31 remainder 0

31 ÷ 2 = 15 remainder 1

15 ÷ 2 = 7 remainder 1

7 ÷ 2 = 3 remainder 1

3 ÷ 2 = 1 remainder 1

1 ÷ 2 = 0 remainder 1

62₁₀ = **111110₂**

Step 2: Convert fractional part.

0.75 × 2 = 1.50 → 1

0.50 × 2 = 1.00 → 1

0.75₁₀ = **0.11₂**

So, 62.75₁₀ = **111110.11₂**

Step 3: Normalize.

111110.11 = **1.1111011 × 2⁶**

Step 4: Determine fields.

• **Sign bit (S):** 0 (positive)

• **Exponent (E):** 5 + 127 = 132 = **10000100₂**

• **Mantissa (M):** 11110110000000000000000 (23 bits, fractional part after leading 1)

Step 5: Final 32-bit representation.

0 10000100 111101100000000000000000 Hexadecimal: **0x427B0000**

4. Define universal gate. Explain Universal gates with their graphical symbol, algebraic expression, truth table, and Venn diagram.

[1 + 4]

Answer: Universal Gate: A universal gate is a logic gate that can be used to implement any Boolean function without requiring any other type of gate. By combining only universal gates, we can construct AND, OR, and NOT operations, and therefore any digital circuit. The two universal gates are **NAND** and **NOR**.

1. NAND Gate (NOT-AND):

Graphical Symbol: An AND gate symbol with a small inversion bubble (circle) at the output.

Algebraic Expression: $Y = (A \cdot B)'$

Truth Table:

ABA·BY = (A·B)' 0001 0101 1001 1110

Venn Diagram Description: Two overlapping circles A and B. The NAND output represents the entire universal set EXCEPT the intersection region ($A \cap B$). The shaded region covers everything outside the intersection.

Implementing Basic Gates with NAND:

- **NOT:** Connect both inputs together: $Y = (A \cdot A)' = A'$
- **AND:** NAND followed by NOT (single-input NAND): $Y = ((A \cdot B)')' = A \cdot B$
- **OR:** Using De Morgan's theorem: $Y = (A' \cdot B')' = A + B$

2. NOR Gate (NOT-OR):

Graphical Symbol: An OR gate symbol with a small inversion bubble (circle) at the output.

Algebraic Expression: $Y = (A + B)'$

Truth Table:

ABA+BY = (A+B)' 0001 0110 1010 1110

Venn Diagram Description: Two overlapping circles A and B. The NOR output represents the region outside both circles (complement of $A \cup B$). The shaded region is only the area exterior to both A and B.

Implementing Basic Gates with NOR:

- **NOT:** Connect both inputs together: $Y = (A + A)' = A'$
- **OR:** NOR followed by NOT: $Y = ((A + B)')' = A + B$
- **AND:** Using De Morgan's theorem: $Y = (A' + B')' = A \cdot B$

5. Define decoder. Explain binary to octal converter with block diagram, truth table and logic diagram.

[1 + 4]

Answer: Decoder: A decoder is a combinational logic circuit that converts binary information from 'n' input lines to a maximum of 2^n unique output lines. Only one output is active (HIGH) at a time for each input combination. Decoders are used in memory addressing, data demultiplexing, and code conversion.

Binary to Octal Converter (3-to-8 Decoder):

Since octal digits range from 0 to 7 (8 states), a 3-bit binary input is required. A 3-to-8 line decoder performs this conversion.

Block Diagram:

+-----+ A2 ---| |--- Y7 (111) A1 ---| 3-to-8 Decoder |--- Y6 (110) A0 ---| |--- Y5 (101) | (Binary to |--- Y4 (100) E ---| Octal) |--- Y3 (011) | |--- Y2 (010) | |--- Y1 (001) | |--- Y0 (000) +-----+ Inputs: A2, A1, A0 (3-bit binary). Enable: E. Outputs: Y0–Y7 (active HIGH).

Truth Table:

EA2A1A0Y7Y6Y5Y4Y3Y2Y1Y0	0XXX00000000	100000000001	100100000010	101000000100
101100001000	110000010000	110100100000	111001000000	111110000000

Logic Diagram: Each output Y_i is generated by a 3-input AND gate with appropriate inputs either direct or inverted:

- $Y_0 = E \cdot A_2' \cdot A_1' \cdot A_0'$
- $Y_1 = E \cdot A_2' \cdot A_1' \cdot A_0$
- $Y_2 = E \cdot A_2' \cdot A_1 \cdot A_0'$
- $Y_3 = E \cdot A_2' \cdot A_1 \cdot A_0$
- $Y_4 = E \cdot A_2 \cdot A_1' \cdot A_0'$
- $Y_5 = E \cdot A_2 \cdot A_1' \cdot A_0$
- $Y_6 = E \cdot A_2 \cdot A_1 \cdot A_0'$
- $Y_7 = E \cdot A_2 \cdot A_1 \cdot A_0$

Each AND gate receives the three address lines; inverters provide the complemented inputs where needed.

6. Simplify the Boolean function $F(w, x, y, z) = \Sigma(0, 2, 4, 6, 8, 10, 12, 14)$ and don't care conditions $d(w, x, y, z) = \Sigma(1, 3, 9, 11)$ using K-Map method for both SOP and POS form.

[2.5 + 2.5]

Answer: Given:

$$F(w, x, y, z) = \Sigma(0, 2, 4, 6, 8, 10, 12, 14)$$

$$d(w, x, y, z) = \Sigma(1, 3, 9, 11)$$

K-Map for SOP (Sum of Products):

Construct the 4-variable K-map with wx as rows and yz as columns:

yz wx 00 01 11 10 00 1 d d 1 01 1 0 0 1 11 1 0 0 1 10 1 d d 1 (1 = minterm, 0 = maxterm/remaining, d = don't care)

Observation: All minterms (1s) occur where $z = 0$. The don't cares (d) all have $z = 1$.

Grouping all eight 1-cells together: they span all rows (w, x vary) and columns yz = 00 and 10 (y varies, $z = 0$). The only constant variable is $z = 0$.

Simplified SOP: $F = z'$

K-Map for POS (Product of Sums):

For POS, we group the 0-cells and treat don't cares as 0s where beneficial. The cells with $z = 1$ (positions 1, 3, 5, 7, 9, 11, 13, 15) can be grouped as a single block of eight by treating don't cares 1, 3, 9, 11 as zeros.

yz wx 00 01 11 10 00 1 d d 1 01 1 0 0 1 11 1 0 0 1 10 1 d d 1 This entire group has $z = 1$ for all its cells (w, x, and y vary). Therefore, the resulting sum term contains only z' .

Simplified POS: $F = z'$

(Note: Both SOP and POS reduce to the same simplest expression $F = z'$ because the function depends only on the variable z.)

7. Define Multiplexer. Explain 4:1 multiplexer with its block diagram, truth table and logic diagram.

[1 + 4]

Answer: Multiplexer (MUX): A multiplexer is a combinational circuit that selects one of several input lines and forwards the selected input to a single output line. The selection is controlled by a set of select lines. It is also called a **data selector**. An N:1 multiplexer has N inputs, $\log_2 N$ select lines, and 1 output.

4:1 Multiplexer:

A 4:1 MUX has 4 data inputs (I_0, I_1, I_2, I_3), 2 select lines (S_1, S_0), and 1 output (Y).

Block Diagram:

```

+-----+ I0 ----> | I1 ----> | 4:1 MUX |----> Y I2 ----> | I3 ----> | S1 S0 | | | S1 ----> | S0 ----> | |
+-----+
  
```

Truth Table:

$S_1 S_0$	Y
00	I_0
01	I_1
10	I_2
11	I_3

Logic Expression:

$$Y = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3$$

Logic Diagram:

The circuit consists of:

- Four 3-input AND gates: each receives one data input and the appropriate select line combination (direct or inverted).
- Two NOT gates to generate S_1' and S_0' .
- One 4-input OR gate to combine the outputs of all four AND gates.

```

I0 ----+----[AND]----+ | S1'S0' | I1 ----+----[AND]----+ +-----+ | S1'S0 +----> | I2 ----+----[AND]----+ | OR |----> Y |
S1S0' +----> | I3 ----+----[AND]----+ +-----+ S1S0 (NOT gates for S1 and S0 are implied before the AND
inputs.)
  
```

8. Write Short Notes on (Any two): a) Parallel Adder b) PLA c) State Diagram

[2.5 + 2.5]

Answer: a) Parallel Adder:

A parallel adder is a digital circuit that adds two multi-bit binary numbers simultaneously. Unlike a serial adder which processes one bit at a time, a parallel adder uses multiple full adders connected in cascade (ripple-carry adder) or with carry-lookahead logic.

Ripple-Carry Parallel Adder:

- For n-bit addition, n full adders (FAs) are required.
- The carry-out from one stage becomes the carry-in of the next (more significant) stage.
- Advantage: Simple structure.
- Disadvantage: Carry propagation delay increases with the number of bits.

4-Bit Parallel Adder Structure:

$A_3B_3 \rightarrow FA_3 \rightarrow S_3 \wedge C_3 \mid A_2B_2 \rightarrow FA_2 \rightarrow S_2 \wedge C_2 \mid A_1B_1 \rightarrow FA_1 \rightarrow S_1 \wedge C_1 \mid A_0B_0 \rightarrow FA_0 \rightarrow S_0 \ C_0=C_{in}$

b) PLA (Programmable Logic Array):

A PLA is a type of programmable logic device used to implement combinational logic circuits. It consists of a programmable AND array followed by a programmable OR array. Unlike PAL (where only the AND array is programmable), both arrays in a PLA can be programmed.

Structure:

- **Input Buffer:** Provides true and complemented inputs.
- **Programmable AND Array:** Generates product terms (minterms) by selecting appropriate inputs.
- **Programmable OR Array:** Combines product terms to form sum-of-products expressions.
- **Output Stage:** May include flip-flops and inverters.

Advantages:

- High flexibility — any combinational function can be implemented.
- Multiple outputs share the same product terms.
- More efficient than ROM for many functions.

c) State Diagram:

A state diagram is a graphical representation of a sequential circuit that shows all possible states, transitions between states, and the inputs/outputs that cause transitions. It is a fundamental tool in the design and analysis of finite state machines (FSMs).

Components:

- **States:** Represented by circles (nodes), labeled with state names/values.
- **Transitions:** Directed arrows showing movement from one state to another.
- **Inputs/Outputs:** Labels on arrows (e.g., input/output).

Types:

- **Mealy Machine:** Output depends on both current state and input.
- **Moore Machine:** Output depends only on current state.

State diagrams are used in designing counters, sequence detectors, and control units of digital systems.

Group C

Attempt any TWO questions. [2×10 = 20]

9. Explain JK and T Flip-flops with their logic diagram, graphical symbol, characteristic table, characteristic equation and excitation table.

[5 + 5]

Answer: 1. JK Flip-Flop:

The JK flip-flop is a universal flip-flop that eliminates the indeterminate state of the SR flip-flop. It has two inputs J (set) and K (reset). When J=K=1, the output toggles.

Logic Diagram: A JK flip-flop can be built from an SR flip-flop with feedback from outputs to inputs through AND gates, or using a master-slave configuration.

Graphical Symbol:

+-----+ J --| | | JK |-- Q K --| | | Q' CLK-> | +-----+ (The triangle > indicates edge-triggered clock input.)

Characteristic Table:

JK	Q(t)	Q(t+1)	State
00	0	0	No change
00	1	1	No change
01	0	0	Reset
01	1	0	Reset
10	0	1	Set
10	1	1	Set
11	0	1	Toggle
11	1	0	Toggle

Characteristic Equation:

$$Q(t+1) = J \cdot Q'(t) + K' \cdot Q(t)$$

Excitation Table:

Q(t) → Q(t+1) JK 0 → 00 X 0 → 11 X 1 → 0X 1 1 → 1X 0 (X = don't care)

2. T Flip-Flop (Toggle Flip-Flop):

The T flip-flop has a single input T. When T=0, the output remains unchanged; when T=1, the output toggles (complements). It is derived from a JK flip-flop by connecting J and K together.

Graphical Symbol:

+-----+ T --| | | T |-- Q | | CLK-> Q' | +-----+

Characteristic Table:

T	Q(t)	Q(t+1)	State
0	0	0	No change
0	1	1	No change
1	0	1	Toggle
1	1	0	Toggle

Characteristic Equation:

$$Q(t+1) = T \oplus Q(t) = T \cdot Q'(t) + T' \cdot Q(t)$$

Excitation Table:

Q(t) → Q(t+1) T 0 → 00 0 → 11 1 → 01 1 → 10

10. Differentiate between asynchronous and synchronous sequential circuits with example. Draw a block diagram, truth table and timing diagram to store 1001 in 4-bit SIPO register.

[4 + 6]

Answer: Difference between Asynchronous and Synchronous Sequential Circuits:

Aspect	Asynchronous Sequential Circuit	Synchronous Sequential Circuit
Clock	No common clock; inputs change directly affect outputs after propagation delay	All flip-flops are triggered by a common clock signal
Speed	Generally faster (no clock waiting)	Slower due to clocked operation
Design	More complex to design; timing analysis is critical	Easier to design and analyze
States	Can have transient/hazard states	State changes occur only at clock edges
Reliability	More prone to race conditions and glitches	More reliable and stable
Examples	SR latch, asynchronous counters	Flip-flops, synchronous counters, registers

4-bit SIPO (Serial In Parallel Out) Register:

A SIPO register accepts data serially (one bit at a time) and outputs all bits in parallel after all bits have been shifted in.

Block Diagram:

Serial +----+ +----+ +----+ +----+ Input CLK->| FF0|--Q0>| FF1|--Q1>| FF2|--Q2>| FF3|--Q3 | | | | | +----+
+----+ +----+ +----+ ^ ^ ^ ^ Q0 Q1 Q2 Q3 (Parallel Outputs) Four D flip-flops connected in cascade. The same CLK is applied to all flip-flops.

Truth Table for Storing 1001:

The serial input is **1** → **0** → **0** → **1** (MSB first).

Clock PulseSerial InputQ3Q2Q1Q0State Initial-0000Reset 1st ↑10001Shift in 1 2nd ↑00010Shift in 0 3rd ↑00100Shift in 0 4th ↑11001Store 1001

Timing Diagram Description:

CLK: SI: Q0: (1, then 0, then 0, then 1) Q1: (0, then 1, then 0, then 0) Q2: (0, then 0, then 1, then 0) Q3: (0, then 0, then 0, then 1) After the 4th clock pulse, the parallel output Q3Q2Q1Q0 = **1001**.

11. Define counter. Write a procedure to design a counter circuit. Design MOD-8 up counter.

[1 + 2 + 7]

Answer: Counter: A counter is a sequential logic circuit that goes through a predetermined sequence of states upon the application of input pulses. Counters are used for counting events, generating timing signals, frequency division, and digital clock circuits. They are classified as asynchronous (ripple) counters and synchronous counters.

Procedure to Design a Counter Circuit:

1. **Determine the number of flip-flops (n):** For a MOD-N counter, find the smallest n such that $2^n \geq N$.
2. **Draw the state diagram:** Show all N states and transitions between them.
3. **Construct the state table:** List present states, next states, and required flip-flop inputs.
4. **Choose flip-flop type:** Usually JK or T flip-flops.
5. **Determine flip-flop inputs:** Use excitation tables to find J, K (or T) inputs for each state transition.
6. **Simplify using K-maps:** Derive minimal Boolean expressions for each flip-flop input.
7. **Draw the logic diagram:** Connect flip-flops and required gates according to the simplified expressions.

Design of MOD-8 Up Counter:

A MOD-8 counter has 8 states (0 to 7), so it requires **n = 3** flip-flops (since $2^3 = 8$).

State Diagram:

000 --> 001 --> 010 --> 011 --> 100 --> 101 --> 110 --> 111 --> 000 (0) (1) (2) (3) (4) (5) (6) (7)

State Table (using T flip-flops):

Present State	Q2	Q1	Q0	Next State	Q2	Q1	Q0	T2	T1	T0
000	0	0	0	001	0	0	1	0	0	1
001	0	0	1	010	0	1	0	0	1	0
010	0	1	0	011	0	1	1	0	0	0
011	0	1	1	100	1	0	0	1	0	0
100	1	0	0	101	1	0	1	0	0	0
101	1	0	1	110	1	1	0	0	0	0
110	1	1	0	111	1	1	1	0	0	0
111	1	1	1	000	0	0	0	1	1	1

K-Map Simplification:

From the state table and T flip-flop excitation ($T = Q \oplus Q_{next}$, or $T=1$ when toggle needed):

- **T0:** Toggles every clock cycle → **T0 = 1**
- **T1:** Toggles when Q0 = 1 → **T1 = Q0**
- **T2:** Toggles when Q0 = 1 AND Q1 = 1 → **T2 = Q1 · Q0**

Logic Diagram:

+-----+ +-----+ +-----+ T2 --| T2 | T1 --| T1 | T0 --| T0 | (Q1Q0)| FF2 || FF1 || FF0 || Q2 Q2'| | Q1 Q1'| | Q0 Q0'| +---^---+ +---^---+ +---^---+ || CLK -----+-----+ | Q0 -----+-----> T1 input Q1 ----+ || +---[AND]---> T2 input Q1·Q0

All three T flip-flops share the same **CLK** signal (synchronous operation). T0 is permanently connected to logic 1 (Vcc). T1 is connected to Q0. T2 is driven by an AND gate with inputs Q1 and Q0.

Operation: The counter increments by 1 on each clock pulse, cycling through 000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000. The output frequency at Q0 is half the clock, at Q1 is one-fourth, and at Q2 is one-eighth of the clock frequency.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.